

Software Design Document

for

QTC Workflow Queue (WFQ)

Version 1.8

Prepared by **Dylan Huang**
Laila Velasquez
Samuel Acevedo
Haik Oganessian
Ruben Flores
Slok Patel
Edwin Rojas
Jesus Ojeda
Alexander Diaz
Zhen Zhao

QTC

10/17/2025

Table of Contents.....	<pg # 2-3>
Revision History.....	<pg #4>
1. Introduction.....	<pg #5>
1.1. Purpose.....	<pg #5>
1.2. Document Conventions.....	<pg #5>
1.3. Intended Audience and Reading Suggestions.....	<pg #5>
1.4. System Overview.....	<pg #5-6>
Design Considerations.....	<pg #7>
2.1. Assumptions and dependencies.....	<pg #7-8>
2.2. General Constraints.....	<pg #8>
2.3. Goals and Guidelines.....	<pg #8-9>
2.4. Development Methods.....	<pg #10>
Architectural Strategies.....	<pg #10-12>
System Architecture.....	<pg #13-14>
4.1.	<pg #13-14>
4.2.	<pg #16>
Policies and Tactics.....	<pg #16>
5.1. Specific Products Used.....	<pg #16>
5.2. Requirements traceability.....	<pg #16>
5.3. Testing the software.....	<pg #16>
5.4. Engineering trade-offs.....	<pg #16>
5.5. Guidelines and conventions.....	<pg #16>
5.6. Protocols.....	<pg #16>
5.7. Maintaining the software.....	<pg #17>
5.8. Interfaces.....	<pg #17>
5.9. System's deliverables.....	<pg #17>
5.10. Abstraction.....	<pg #17-18>
Detailed System Design.....	<pg #19>
6.x Name of Module.....	<pg #19>
6.x.1 Responsibilities.....	<pg #19>
6.x.2 Constraints.....	<pg #19>
6.x.3 Composition.....	<pg #19-20>
6.x.4 Uses/Interactions.....	<pg #20>
6.x.5 Resources.....	<pg #20>
6.x.6 Interface/Exports.....	<pg #20-21>
Detailed Lower level Component Design	
7.x Name of Class or File.....	<pg #22>
7.x.1 Classification.....	<pg #22>
7.x.2 Processing Narrative(PSPEC).....	<pg #22>
7.x.3 Interface Description.....	<pg #22-23>
7.x.4 Processing Detail.....	<pg #23>
7.x.4.1 Design Class Hierarchy.....	<pg #23>
7.x.4.2 Restrictions/Limitations.....	<pg #23>
7.x.4.3 Performance Issues.....	<pg #23>
7.x.4.4 Design Constraints.....	<pg #23>
7.x.4.5 Processing Detail For Each Operation.....	<pg #24>

User Interface

- 8.1. Overview of User Interface.....<pg #25>
- 8.2. Screen Frameworks or Images.....<pg #25-26>
- 8.3. User Interface Flow Model.....<pg #26-27>

Database Design

Requirements Validation and Verification.....<pg #32-37>

Glossary.....<pg #38>

References.....<pg # 39-40>

Revision History

Name	Date	Reason For Changes	Version
All group members	10/17/2025	Initial Document	1.0
Slok Patel	11/4/2025	Finalized Section 2, Removed any API info.	1.6
Ruben Flores	10/22/25	Section 3	1.2
Edwin Rojas	10/22/2025	Section 1 Introduction Section 4 System Architecture	1.2
Samuel Acevedo	10/23/2025	Section 8: Database Design completion	1.3
Ruben Flores	10/23/2025	Section 4, 6, and 7, 9	1.4
Alexander Diaz	10/23/2025	Section 9 User Interface	1.5
Laila Velasquez	10/23/2025	Section 10: Requirements Validation and Verification and Table of Contents	1.5
Dylan Huang	10/23/2025	Section 12: References	1.5
Haik Oganessian	10/22/2025	Section 5: Policies and Tactics Section 11: Glossary	1.5
Edwin Rojas	11/4/2025	Section 1 Introduction Revisions Section 4 System Architecture Revisions	1.6
Samuel Acevedo	11/5/2025	Section 12.2: Regulatory & Policy References Revision	1.7
Zhen Zhao	11/5/2025	Section 12.2 Regulatory Revisions	1.7
Edwin Rojas	11/5/2025	Section 10 Requirements Validation and Verification Revisions	1.7

Ruben Flores	11/5/2025	1.2 " Existing WFQ Database (Direct Access)" mentioned existing database and to not alter 3.11 - Added existing architecture info	1.8
Samuel Acevedo	11/12/2025	Removed Section 12.2 and added more into Section 2.2 Section 3.4 removed api, changed with direct application transaction call	1.8
Laila Velasquez	11/12/2025	Section 10 edited table Remove API calling throughout the document	1.8
Edwin Rojas	11/12/2025	Section 1 Introduction updated component descriptions	1.8
Dylan Huang	11/14/2025	Removed Business Logic Mentions in 6.2	1.8
Jesus Ojeda	11/17/2025	Revised section 6.4.6 Interface/Exports for clarity Revised Section 4.1 Level 0 DFD description and updated image formatting	1.8
Jesus Ojeda	11/18/2025	Revised formatting for section 7 and section 8 to match pre-existing formatting	1.8

1. Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to define the technical design, architectural components, and implementation strategies for the (New) QTC Workflow Queue (WFQ) Administrator Application.

This document serves as the primary technical specification for the development team. It guides the implementation of a new, monolithic ASP.NET Core MVC application that performs administrative functions through direct data-level interaction with the (Existing) WFQ database to meet the requirements defined by Leidos QTC Health Services.

1.2 Document Conventions

This document follows standard **Software Design Document (SDD)** conventions for clarity and consistency.

- **Code snippets, file names, commands, and SQL statements** are formatted in *monospace font* (e.g., `CREATE TABLE`, `Program.cs`).
- **System components and modules** are referred to using **bold text** (e.g., **Admin Dashboard**, **Workflow Queue**).
- **User stories** and **requirements of identifiers** (e.g., *REQ-1.1*) are listed where applicable to reference design traceability.
- **Diagrams** (such as data flow or sequence diagrams) are labeled and captioned for cross-reference in other sections of the document.

1.3 Intended Audience and Reading Suggestions

This document is intended for multiple roles within the project, including:

- **Developers:** To understand the system architecture, database schema, and technical design necessary for implementing and extending features.
- **Testers/QA Engineers:** To identify system workflows, interfaces, and data flows that require verification and validation during testing.
- **Project Managers and Stakeholders:** To review the technical feasibility and alignment of the system design with business objectives.
- **Technical Writers:** To create or update user documentation based on finalized design components.

1.4 System Overview

The project entails the development of a single, (New) web application designed to provide administrative control over the (Existing) QTC Workflow Queue (WFQ) system. The new

application's sole integration point is a direct, data-level connection to the existing WFQ MS SQL Server database. The system is organized into two major components:

1. **New Administrator Application (New):** An ASP.NET Core MVC application. It contains the server-side rendered User Interface (UI) using Razor Pages, the business-level service layer, and the data access layer required to manage the workflow system. Its sole purpose is to provide the administrative capabilities outlined in the "Workflow Queue Administrator Requirements" document (R2).
2. **WFQ Database (Existing):** The persistent Microsoft SQL Server data store used by the QTC Workflow Queue system. The database schema, its tables, and its stored procedures will not be modified by this project.

IN SCOPE:

- Designing and building a new ASP.NET Core MVC web application hosted on IIS
- Implementing a server-side rendered user interface
- Implementing all administrative features as specified in the "Workflow Queue Administrator Requirements" document
- Implementing a simple, session-based authorization check based on a URL query string flag provided in the initial request

OUT OF SCOPE:

- Modifying the (Existing) Workflow database schema or its stored procedures
- Making any changes to the existing User & Manager UI

2. Design Considerations

This section describes the major design issues, assumptions, dependencies, and constraints that influence the structure and implementation of the QTC Workflow Queue (WFQ) Administrator Module.

2.1 Assumptions and Dependencies

Our design and implementation rely on the following assumptions and dependencies:

Software/Hardware Dependencies:

- **Dependency on WFQ Database Schema (Existing):** The design is entirely dependent on the schema, constraints, and stored procedures of the (Existing) WFQ SQL Server database. All data operations performed by the (New) application must conform to this existing, unchangeable structure.
- **Direct Database Interaction (No API):** The application communicates directly with the SQL Server database through server-side data access logic (e.g., ADO.NET or Entity Framework). There is **no** intermediary API layer between the UI and database. All database transactions are handled securely from the server-side C# code running within the ASP.NET framework.
- **Dependency on Microsoft Environment:** The solution is dependent on the QTC's Microsoft-based technology stack, including:
 - **Operating System:** Windows Server
 - **Web Server:** Internet Information Services (IIS)
 - **Database:** Microsoft SQL Server
 - **Framework:** .NET Framework / ASP.NET
- **Dependency on URL-based Authorization Flag:** Administrative access will be granted through a query string parameter (e.g., `?is_admin=true`) appended by the hosting environment. The system depends on this flag for enabling administrative functions, with no external authentication module required.

Organizational Dependencies:

- **Stable Requirements:** We assume that the "Workflow Queue Administrator Requirements" document (R2) represents a stable set of functional requirements for the (New) Admin Application, and that major changes will be minimal.

Assumptions:

- The existing Workflow MS SQL Server database schema is stable and will not be changed during the project's lifecycle.
- The application will be deployed to a QTC-managed Windows Server environment with IIS and .NET Core hosting enabled.
- The team possesses, or can acquire, the necessary skills in C#, ASP.NET Core MVC, Entity Framework Core, and T-SQL.
- **Key Challenges and Risks:**
 - **Performance Impact:** Administrative database operations (reads and writes) could potentially impact the performance of the live QTC workflow system that shares the database.
 - **Data Integrity:** Direct database access carries the risk of data corruption if operations are not handled atomically.

2.2 General Constraints

The design is affected by the following global constraints:

- **Technology Stack Constraint:** The project is constrained to the existing QTC technology stack: C#, ASP.NET Core MVC, Entity Framework Core, and MS SQL Server.
- **Direct Database Access Only:** The application's sole integration point is a direct connection to the MS SQL Server. No intermediary APIs will be used for data access.
- **Immutable Database:** The WFQ database schema, including all tables, views, and stored procedures, is considered immutable. This project will not make any modifications to the database structure. The application's role is to read from and write data to the existing tables, not to alter them.
- **Simple Query String Flag:** User authorization is constrained to checking for a URL query string parameter (e.g., ?is_admin=true) appended to the URL by the hosting infrastructure.
- **Standards Compliance (HIPAA/HITECH):** As the application will interact with a database containing Protected Health Information (PHI) and Personally Identifiable Information (PII), the design must be compliant with the Health Insurance Portability and Accountability Act (HIPAA) and HITECH Act. This is achieved by:
 - Implementing strict role-based access control via the specified authorization method.
 - Ensuring all data in transit is encrypted.

- Leveraging the existing database's audit fields (CreatedBy, CreatedDate, etc.) for all data modifications to maintain a clear audit trail.
- **Security Requirements:**
 - All communication between the user's browser and the web server must be encrypted over HTTPS/TLS
 - Access to administrative functions must be enforced server-side
 - Database credentials and connection strings must be securely stored within the server configuration
- **Business Rule Constraints:** The Admin UI must follow sponsor-defined business rules — for example, workflows cannot be deleted via the user interface (Admin Requirements, item 4c).
- **User Interface Constraints:** The Admin UI must visually and functionally integrate with the existing WFQ web application, adhering to QTC and Leidos branding standards (logo, color palette, and layout).

2.3 Goals and Guidelines

The project design is guided by the following goals and principles:

- **Primary Goal — Deliver a Complete Administrator Interface:** Build an intuitive, secure, and functional Admin UI that allows authorized users to create, modify, and manage workflows directly through the WFQ system.
- **Goal — Adherence to Professional Coding Standards:** Follow Microsoft C# coding conventions and QTC's internal best practices, emphasizing maintainable, secure, and well-documented code.
- **Guideline — Maintainability and Extensibility:** The solution should be modular and maintainable, with clean server-side logic, reusable components, and well-commented source code to support future enhancements.
- **Guideline — Consistency:** The Admin module should look, behave, and perform like a natural extension of the existing WFQ application, ensuring visual and functional consistency across modules.

2.4 Development Methods

This project uses a Hybrid Agile approach, combining structured documentation with iterative development cycles. This method allows flexibility for sponsor feedback while maintaining clear deliverables.

- **Iterative Development:** Work is divided into short development cycles that refine both the **frontend** (Admin UI) and **backend (database transactions)**. Each iteration includes design, implementation, and testing.
- **Stakeholder Collaboration:** The project team meets regularly with the QTC sponsor to demonstrate progress, gather feedback, and align priorities.
- **Requirements Backlog:** All tasks are derived from the “Required Improvements” and “Workflow Queue Administrator Requirements” documents, prioritized with input from the sponsor.
- **Focus on Deliverables:** The team will produce both academic deliverables (SRS, SDD) and sponsor deliverables (working code, UI mockups, documentation) for review and feedback after each major milestone.

3. Architectural Strategies

3.1 Technology Stack (language, framework, runtime)

The system uses C# with ASP.NET MVC on .NET Framework 4.6.1 or higher. MS SQL Server provides the database. Entity Framework is used with stored procedures. IIS on Windows Server hosts the application.

3.2 Reuse & integration strategy

Focus in on direct data-level interaction with the existing QTC ecosystem. The application reuses the existing, canonical WFQ SQL Server database schema as its single source of truth. There is no reuse or integration with any other application-level components (like APIs or UI libraries).

3.3 Strategy for program Versions

No REST API design supports versioning. Database schema changes are managed with versioning scripts. Service layer is modular to support upgrades. REST endpoints provide stable contracts for external systems.

3.4 UI & interaction paradigms

Modern, web-based user interface (UI) for administrative tasks. The UI is a "thin client" that does not contain business logic. It interacts exclusively with its own dedicated backend via direct application transaction calls (asynchronous HTTP requests) to fetch data and trigger operations. All user actions, from viewing workflows to deleting steps, are funneled through these direct transaction calls to the application's backend.

3.5 Error detection, logging, and recovery

User Input Validation (Preventing Errors):

- When an administrator fills out a form (like "Create New Workflow") and forgets a required field or enters invalid data, the form will not submit. Instead, the message "Workflow Name is required" will appear next to the field they need to fix. This will be handled on both the client-side and the server-side before any data is sent to the database.

Diagnostic Logging (Finding the Cause):

- The exception handling mechanism will log the full exception details, including the error message, stack trace, and the time it occurred, to a text file on the server. This allows developers to diagnose and fix bugs without needing to reproduce them.

Unexpected System Errors (Graceful Failure):

- The backend will use global exception handling (a "try-catch" block around all operations). If an unhandled exception occurs, it will be caught, logged, and the user will be redirected to a static error page.

3.6 Data storage & persistence strategy

MS SQL Server is the canonical data store. Critical operations execute via stored procedures. History tables (TaskHistory, WorkflowLog) retain past state for audit purposes.

3.7 Distribution, concurrency, and synchronization

The existing WFQ database schema includes a VerCol (rowversion/timestamp) column on key tables like Task. When updating a record, our UPDATE statement will include a WHERE clause that checks if the VerCol value is the same as it was when we first read the data. If the rowversion is different (meaning another process changed it), the UPDATE will affect zero rows, and our code will interpret this as a concurrency conflict, sending the appropriate error message back to the UI.

3.8 Communication mechanisms & security

All communication between the administrator's web browser and the application's web server will be secured using HTTPS/TLS. The backend will enforce authorization for all actions by inspecting a URL query string parameter (is_admin=true). This security check ensures that only authenticated and authorized sessions can perform administrative functions. Payloads between the UI and backend will use JSON format.

3.9 Resource management & operational strategies

IIS application pool runs under a constrained identity. Database connections use ADO/EF pooling. Maintenance, backups, and monitoring of logs and metrics are performed regularly.

3.10 Memory management & performance strategies

Controllers handle minimal processing. Heavy computation and transient data are processed in the service layer. Server-side pagination (OFFSET-FETCH) limits dataset size. Master data caching reduces database calls.

3.11 Generalized approaches to control & governance

Business rules reside in stored procedures and service layer logic. All state transitions are audited in TaskHistory and WorkflowLog, which are part of the existing architecture and will not be modified.

4. System Architecture

This section provides a high-level overview of the new WFQ Administrator Application's architecture. The system is designed as a self-contained web application that interacts directly with a shared database, separating it from the existing live WFQ application.

4.1 Level 0 Data Flow Diagram (DFD)

The Level 0 DFD illustrates the application's context and its interaction with external components. The system is self-contained and only interacts directly with the QTC Administrator and the existing Workflow Database.

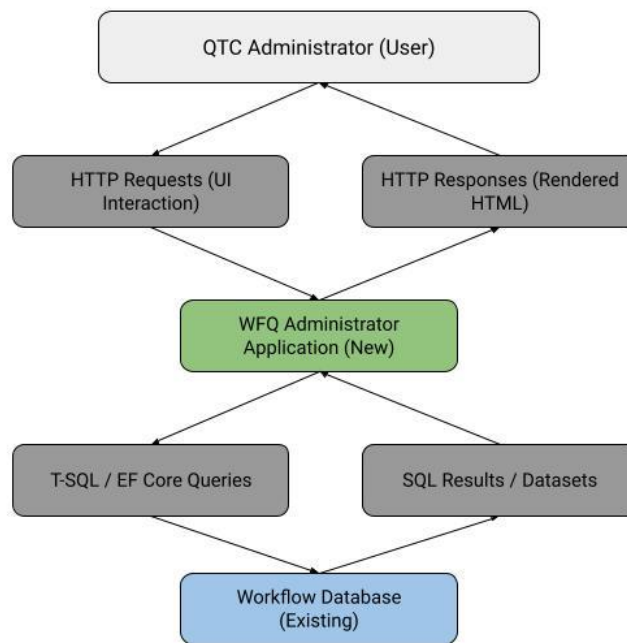


Figure 4.1: Level 0 Data Flow Diagram

4.2 Level 1 Data Flow Diagram (DFD)

The Level 1 DFD decomposes the WFQ Administrator System into three distinct layers to ensure separation of concerns and maintainability.

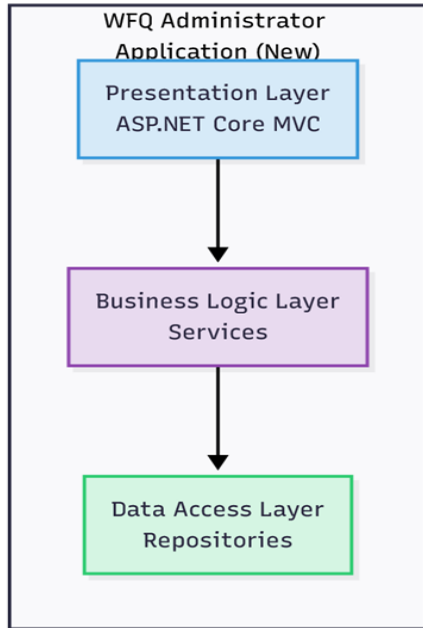


Figure 4.2: Level 1 Data Flow Diagram

5. Policies and Tactics

This section describes specific design policies and engineering tactics adopted for the WFQ Administrator Application. These policies influence implementation details and interface behavior but do not fundamentally alter the system's architecture.

5.1 Choice of which specific products used

- **IDE:**
Visual Studio 2022
- **Web Framework:** ASP.NET Core 7.0 MVC
- **Database:** Microsoft SQL Server
- **Database Management:** SQL Server Management Studio (SSMS) 19
- **Web Server:** Internet Information Services (IIS) 10.0
- **Source Control:** Git, hosted on GitHub

5.2 Plans for ensuring requirements traceability

To ensure all sponsor requirements are met and verified, a Requirements Traceability Matrix (RTM) is maintained in Section 10 of this document.

- **Completeness:** It maps every requirement from the Workflow Queue Administrator Requirements document (R2) to the specific design modules, lower-level components, and verification tests that implement and validate them.
- **Verification:** It serves as a checklist for the QA team to ensure that a test case exists for every single requirement.

- **5.3 Plans for testing the software**

- **Unit Testing:** We will write tests that mock the Data Access Layer to verify all business logic (e.g., the complex "safe delete" for a workflow step) in isolation, without needing a database.
- **End-to-End (E2E) Manual Testing:** The QA team will perform manual tests based on the use cases and the RTM (Section 10). This involves running the full application locally and testing all user-facing functionality, from creating a workflow to verifying that user-friendly error messages appear for concurrency conflicts.

- **5.4 Coding guidelines and conventions**

- For the duration of this project, we will adhere to the Microsoft C# Coding Conventions provided by the sponsor in their initial project documentation. This includes guidelines for naming, layout, commenting, and language usage.

5.5 The choice of a particular algorithm or programming idiom (or design pattern) to implement portions of the system's functionality

- Search features
- Coding convention guideline compliance

5.6 Plans for maintaining the software

- Providing accurate documentation, comments, and following coding conventions in order to make sure future developers have an easier time working on the application.

5.7 Interfaces for end-users, software, hardware, and communications

- Admin panel including buttons, dashboards, screens for displaying different information and accessibility features.
- All communication is done through encrypted protocols/channels.

5.8 How to build and/or generate the system's deliverables (how to compile, link, load, etc.)

- **Prerequisites:** A developer workstation must have the .NET 7 SDK and Visual Studio 2022 installed.
- **Process Request (Build):** The application will be built using the standard dotnet build command or via the Visual Studio IDE. This compiles all C# code into a set of DLLs.
-

6. Detailed System Design

6.1 Presentation Layer Module

6.1.1 Responsibilities

- Render HTML and present data to the QTC Administrator in a user-friendly web interface
- Capture user input from HTML forms, such as creating a new workflow or editing a step
- Perform basic client-side validation (e.g., checking for required fields) for immediate user feedback
- Invoke methods on the Business Logic Service Module (6.2) to perform application-specific operations and business rule processing. It does not contain any business logic itself.
- Display system status, confirmations, and user-friendly error messages returned from the service layer

6.1.2 Constraints

- **Thin Controller:** The controllers within this module must contain minimal logic. Their responsibility is strictly to manage the web request/response cycle. All complex business logic must be delegated to the service layer.
- **Branding:** The UI's appearance, including colors, fonts, and logos, must adhere to the Leidos corporate branding guidelines
- **Browser Compatibility:** The UI must be functional and render correctly on the latest versions of modern web browsers (e.g., Chrome, Edge, Firefox).

6.1.3 Composition

- **Workflow Dashboard Component:** Displays the main paginated and searchable list of all workflows.
- **Workflow Details Component:** Displays the steps, tasks, and history for a single selected workflow.
- **Create/Edit Modal Components:** Reusable dialog forms for creating and editing workflows and steps.
- **Confirmation Dialog Component:** A generic modal used to confirm destructive actions like deleting a step.

- **Notification/Toast Service:** A UI element for displaying non-blocking success or error messages (e.g., "Workflow saved successfully").

6.1.4 Uses/Interactions

- **Used By:** The QTC Administrator (the end-user) interacts directly with this module through their web browser.
- **Uses:** Authorization Module (6.4) declaratively via the [Authorize] attribute to protect controller actions and ensure the user has admin privileges.

6.1.5 Resources

- This module consumes CPU and memory resources on the end-user's client machine (i.e., within their web browser).

6.1.6 Interface/Exports

- **Exports:** A fully rendered, interactive Graphical User Interface (GUI).
-

6.2 Business Logic Service Module (BLL)

6.2.1 Responsibilities

- To implement the application's use cases as defined in the requirements by coordinating interactions between controllers, data-access components, and external services.
- To manage workflow-related operations such as validating requests, enforcing allowed state transitions, and ensuring consistent updates to workflow steps, tasks, and related records.

6.2.2 Constraints

- **Technology Agnostic:** This module must have no knowledge of the web layer (HTTP). It should not reference HttpContext or any web-specific constructs.
- **Testability:** The logic must be fully unit-testable without requiring a live web server or database. Dependencies (like the Data Access Module) should be injectable to allow for mocking during tests.

The module is composed of a set of C# service classes, such as WorkflowAdminService.cs, which encapsulate related business logic

- **Used By:** The Presentation Layer Module (6.1).
- **Uses:** It exclusively uses the Data Access Module (6.3).

- This module is the primary consumer of server CPU and memory for processing business rules.

6.2.6 Interface/Exports

- **Exports:** A set of C# classes and public methods that can be injected into and called by the controller layer. For example: Task<ServiceResult> CreateWorkflowAsync(WorkflowCreateModel model).

6.3 Data Access Module (DAL)

6.3.1 Responsibilities

- To encapsulate all database interaction logic.
- To perform all CRUD (Create, Read, Update, Delete) operations against the WFQ database.
- To execute both direct queries (via Entity Framework) and stored procedures as required.
- To manage database connections and transactions to ensure data integrity.
- To handle database-specific errors (e.g., foreign key violations, connection timeouts) and translate them into exceptions that the service layer can understand.
- To implement the concurrency checks using the VerCol (rowversion) column.

6.3.2 Constraints

- **Schema Conformance:** All operations must conform to the existing, unchangeable schema of the WFQ database. The application cannot alter the database schema.
- **No Business Logic:** This layer must not contain any business rules. Its only concern is data persistence.

6.3.3 Composition

- An Entity Framework DbContext class that models the WFQ database tables.
- A set of Repository classes (e.g., WorkflowRepository.cs, TaskRepository.cs) that use the DbContext to perform data operations.

6.4.4 Uses/Interactions

- **Used By:** The Business Logic Service Module (6.2).
- **Uses:** It uses the Microsoft.EntityFrameworkCore library and the underlying ADO.NET database driver to communicate with the SQL Server instance.

6.3.5 Resources

- This module manages a critical server resource: the pool of database connections. It is the component most likely to be affected by database performance, race conditions, and deadlocks.
- **Exports:** A set of C# Repository interfaces (e.g., `IWorkflowRepository`) with methods for data manipulation (e.g., `Task<Workflow> GetByIdAsync(int id)`).

6.4 Authorization Module

6.4.1 Responsibilities

- To determine if the current user is an "Admin."
- To provide a simple, reusable pass/fail mechanism that can be used to protect controller actions in the Presentation Layer.

6.4.2 Constraints

- The authorization logic is limited to inspecting a query string parameter (e.g., `is_admin=true`) provided in the initial URL. It does not perform lookups against any external service.
- This check must be performed at the beginning of the request pipeline, before any business logic is executed.

6.4.3 Composition

- This module will be implemented as an ASP.NET Core Authorization Policy (e.g., a policy named "IsAdmin").

6.4.4 Uses/Interactions

- **Used By:** The Presentation Layer Module (6.1)
- **Uses:** It uses the `HttpContext` object to access the incoming URL and its query string

6.4.5 Resources

- Consumes minimal server CPU

6.4.6 Interface/Exports

- **Exports:** A named ASP.net Core authorization policy ("isAdmin") that can be applied to controllers or specific controller actions using the standard attribute: `[Authorize(Policy = "IsAdmin")]`

This exported policy serves at the modules interface. When applied, it instructs the ASP.NET Core authorization pipeline to validate the request by examining the query

string for the required is_admin=true flag. If the requirement is not met, the request is blocked before reaching any business logic.

- The following flow illustrates where the authorization module participates in the request pipeline:

Detailed Lower-Level Component Design

7.1 Class: WorkflowsController.cs

7.1.1 Classification

C# MVC Controller class within the Presentation Layer Module (6.1).

7.1.2 Processing Narrative (PSPEC) The WorkflowsController handles all workflow-related HTTP requests. It is responsible for:

Rendering views for listing workflows, creating workflows, and editing steps

- Handling form submissions through POST actions
 - Calling IWorkflowAdminService to perform workflow operations
 - Displaying validation messages or redirecting based on service outcomes
- Maintaining the Thin Controller principle (no business logic)

- **7.1.3 Interface Description**

```
[Authorize(Policy = "IsAdmin")]
public class WorkflowsController : Controller
{
    public async Task<IActionResult> Index();
    public IActionResult Create();

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create([FromForm] WorkflowCreateModel model);

    public async Task<IActionResult> EditStep(int workflowId, int stepId);

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> EditStep([FromForm] WorkflowStepUpdateModel model);

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteStep(int workflowId, int stepId);
}
```

- **7.1.4 Processing Detail**

- 7.1.4.1 Design Class Hierarchy**

WorkflowsController : Microsoft.AspNetCore.Mvc.Controller

- 7.1.4.2 Restrictions / Limitations**

- Requires an injected IWorkflowAdminService instance
- Cannot operate without service-layer availability

- 7.1.4.3 Performance Considerations**

- Minimal overhead; performance depends on service layer

- 7.1.4.4 Design Constraints**

Must follow Thin Controller design

- No business logic allowed

7.1.4.5 Processing Detail for Each Operation

- Index: Calls GetAllWorkflowsAsync, maps results, returns view
- Create (POST): Validates model, calls CreateWorkflowAsync, handles success/errors
- EditStep/DeleteStep: Retrieves target records, performs updates/deletions using service layer

• 7.2 Class: WorkflowAdminService.cs

7.2.1 Classification

C# Service class within the Business Logic Service Module (6.2).

7.2.2 Processing Narrative (PSPEC)

The WorkflowAdminService coordinates all workflow-related business logic. Responsibilities include:

- Enforcing workflow rules such as unique workflow names
- Running multi-step operations within database transactions
- Coordinating with repository components

Remaining fully decoupled from UI and data layers

7.2.3 Interface Description

```
public interface IWorkflowAdminService
{
    Task<ServiceResult<WorkflowDto>> CreateWorkflowAsync(WorkflowCreateModel model);
    Task<ServiceResult> UpdateWorkflowStepAsync(WorkflowStepUpdateModel model);
    Task<ServiceResult> DeleteWorkflowStepAsync(int workflowId, int stepId);
}
```

7.2.4 Processing Detail

7.2.4.1 Design Class Hierarchy

WorkflowAdminService : IWorkflowAdminService

7.2.4.2 Restrictions / Limitations

Depends on repository interfaces such as IWorkflowRepository and ITaskRepository

- Requires dependency injection for testability

7.2.4.3 Performance Considerations

- Efficiency depends on repository operations and workflow complexity

7.2.4.4 Design Constraints

- All business logic must reside here

Multi-operation changes require atomic database transactions

7.2.4.5 Processing Detail for Each Operation

- CreateWorkflowAsync: Validates uniqueness, maps model, inserts entity, commits
 1. DeleteWorkflowStepAsync: Runs inside Unit-of-Work, reassigns dependent tasks, deletes step, commits

7.3 Class: WorkflowRepository.cs

7.3.1 Classification

C# Repository class within the Data Access Module (6.3).

7.3.2 Processing Narrative (PSPEC)

The WorkflowRepository handles all CRUD operations related to workflows and workflow steps. It performs:

- Entity Framework queries

Data retrieval and persistence

- Basic database operations without business logic

7.3.3 Interface Description

```

public interface IWorkflowRepository
{
    Task<Workflow?> GetByNameAsync(string name);
    Task<WorkflowStep?> GetStepByIdAsync(int stepId);
    Task AddAsync(Workflow workflow);
    void DeleteStep(WorkflowStep step);
}

```

7.3.4 Processing Detail

7.3.4.1 Design Class Hierarchy

WorkflowRepository : IWorkflowRepository

7.3.4.2 Restrictions / Limitations

Requires valid DbContext and database connectivity

- Limited to existing database schema

7.3.4.3 Performance Considerations

- Query performance affects system responsiveness

Large datasets require indexed queries and pagination

7.3.4.4 Design Constraints

- Must contain no business logic
- All operations must translate efficiently to SQL

7.3.4.5 Processing Detail for Each Operation

- GetByNameAsync
 - return await _context.Workflows
 - .AsNoTracking()
 - .FirstOrDefaultAsync(w => w.WorkflowName

== name);
- AddAsync
 - await _context.Workflows.AddAsync(workflow);
- DeleteStep

- `_context.WorkflowSteps.Remove(step);`

8. Database Design

8.1 Overview

The QTC Workflow Queue (WFQ) database is an existing Microsoft SQL Server relational data store. It supports workflow configuration, task processing, and audit logging. The (New) WFQ Administrator Application interacts with this database through direct data-level access. The schema, tables, and stored procedures are part of the existing system and will not be modified by this project.

The WFQ database enables the following functionality:

- Managing workflows and their associated steps
- Tracking tasks and task assignments
- Managing item metadata and status values
- Recording audit history for workflow and task activity
- Providing flexible key-value storage for items

8.2 Table Descriptions

- Below is an overview of the major tables relevant to the WFQ Administrator Application:
 1. Task
Stores individual tasks created within workflows, including assignment, status, and timestamps.
 2. Workflow
Defines an entire workflow consisting of ordered steps.
 3. WorkflowStep
Represents a step within a workflow. Each step contains ordering information and relationships to tasks.
 4. Status
Defines valid status values used by tasks.
 5. Item
Represents the business object or record being processed through workflows.
 6. ItemStatus
Tracks the current status of an Item.
 7. InfoKey
Stores metadata key definitions for dynamic item attributes.
 8. ItemKeyValue
Stores key-value pairs linked to items, providing extensible metadata support.

8.3 Relationships Summary

- Workflow → WorkflowStep
One-to-Many. Each workflow contains multiple ordered steps.
- WorkflowStep → Task
One-to-Many. Each step may contain multiple tasks.
- Item → Task
One-to-Many. A single item may have multiple tasks across steps.
- Status → Task
One-to-Many. Each task has a single status value.
- ItemStatus → Item
One-to-Many. Each item has one current status.
- Item → **ItemKeyValue** → InfoKey
Supports flexible, schema-independent metadata storage.

8.4 Design Rationale

- Normalization
The schema is normalized (3NF) to reduce redundancy and maintain data consistency.
- Auditability
Many tables include CreatedBy, CreatedDate, UpdatedBy, and UpdatedDate fields for audit and compliance.
- Extensibility
The InfoKey and ItemKeyValue tables provide a mechanism for storing additional data without requiring schema changes.
- Performance
Frequently used fields (e.g., Id, WorkflowStepId, StatusId) are indexed to support efficient querying.
- Scalability
Separation of workflows, items, tasks, and metadata supports distributed processing and reporting across large datasets.

9. User Interface

The user interface (UI) is the primary means by which administrators will interact with the WFQ system to manage and monitor workflows. The design prioritizes clarity, efficiency, and consistency with the Leidos corporate branding. The UI is implemented as a web-based application that interacts with the backend.

9.1 Overview of User Interface

From the user's perspective, the WFQ application provides:

- **Dashboard / Workflow List:** Displays all workflows a user has access to, with sortable columns for Name, Status, Created Date, and actions (Edit/View Steps). This UI interacts with backend to retrieve workflow data and with the **Queue Management Subsystem** for workflow operations.
- **Workflow Step View:** Shows all steps associated with a selected workflow, including StepNumber, StepName, and Actions (Edit/Delete). Interacts with **Queue Management** and **Data Persistence** modules.
- **Task Management Page:** Displays tasks assigned to users, with details like AssignedTo, Status, Step, and Actions (Complete Task / Reassign). Interacts with **Queue Management** and **Authorization** modules.
- **Search & Filter Controls:** Provides a search bar and filtering options for workflows and tasks, sending queries to the **Data Persistence Subsystem**.
- **Admin Interface:** Allows admins to create/update workflows, steps, and tasks. Interacts heavily with **Authorization**, **Queue Management**, and **Data Persistence** subsystems.

The UI provides real-time feedback to users through status indicators, confirmation dialogs, and error messages for invalid actions (e.g., unauthorized access or failed validations).

9.2 Screen Frameworks or Images

This section contains mockups and screenshots of the significant UI screens and components for the new Administrator module.

Workflow Management System
QTC Health Services

Administrator Portal

Workflow Management

Manage and monitor all workflow processes

Name A-Z

Create New Workflow

Name	Description	Status	Actions
Budget Allocation	Quarterly budget review and allocation workflow	Completed	
Content Review Pipeline	Editorial review and approval workflow for content	Completed	
Customer Onboarding	Automated workflow for new customer registration and setup	In Progress	
Employee Offboarding	Complete employee exit process and access revocation	Completed	
Expense Approval	Employee expense submission and approval process	In Progress	
Invoice Processing	Streamlined invoice approval and payment workflow	New	
Product Launch Checklist	Multi-step workflow for coordinating product releases	New	
Security Incident Response	Coordinated response workflow for security incidents	In Progress	
Support Ticket Escalation	Automated escalation for high-priority support tickets	In Progress	
Vendor Onboarding	New vendor registration and compliance verification	New	

Items per page: 10

Page 1 of 1

< Previous

1

Next >

Figure 9.1: Admin Workflow Management Dashboard

Create New Workflow

Fill in the details to create a new workflow.

Name *

Patient Onboarding Process

Description

Defines steps for new patient registration and initial assessment

Cancel

Save Workflow

Figure 9.2: Create/Edit Workflow Dialog

← Back to Workflows

Budget Allocation
Quarterly budget review and allocation workflow

Steps Tasks History

Workflow Steps [+ Create New Step](#)

Step Number	Step Name	Description	Actions
No steps found. Create your first step to get started.			

Figure 9.3: Workflow Step Editor view

9.3 User Interface Flow Model

This section describes the navigation and action flows for a typical administrative user.

Use Case 1: Creating a new Workflow

1. The administrator navigates to the Workflow Management Dashboard
2. The user clicks the "+ Create New Workflow" button
3. A modal dialog appears, presenting a form with fields for "Name" and "Description"
4. The user enters the required information and clicks "Save"
5. The UI submits an HTML form to the /Workflows/Create controller action
6. Upon a successful response, the modal closes, and the workflow list on the dashboard automatically refreshes to include the newly created workflow. An error message is displayed (e.g., the workflow name already exists)

Use Case 2: Editing the Steps of an Existing Workflow

1. From the Workflow Management Dashboard, the administrator locates the desired workflow in the list
2. The user clicks the "View" (eye) icon in the "Actions" column for that workflow
3. The UI navigates to a "Workflow Details" page (or expands a nested view) which displays the list of steps for that workflow, showing columns like "StepNumber," "StepName," and "Actions (Edit/Delete)"
4. To edit a step, the user clicks the "Edit" (pencil) icon next to the desired step
5. A modal dialog appears, pre-populated with the current data for that step
6. The user modifies the data and clicks "Save"
7. The UI submits an HTML form to the /Workflows/EditStep controller action.
8. On success, the modal closes and the list of steps refreshes to show the updated information

Use Case 3: Deleting a Workflow Step

1. From the "Workflow Details" page, the administrator clicks the "Delete" (trash) icon next to a step
2. A confirmation dialog appears, warning the user that this action is irreversible. "Are you sure you want to delete this step?"
3. The user clicks "Confirm"
4. The UI submits an HTML form to the /Workflows/DeleteStep controller action.
5. The backend service executes the required business logic (checking for dependent tasks, creating a new task for the next step if necessary) before deleting the record
6. On success, the list of steps refreshes, and the deleted step is removed from the view

10. Requirements Validation and Verification

Requirement ID	Requirement from Admin Docs	Component Modules	UI Element(s) / Low-Level Components	Verification Method (Test)
ADM-1	Users must be authorized as "Admin" role to access the UI.	Authorization Module (6.5); Web Controller Module (6.2)	[Authorize(Policy="IsAdmin")] attribute on controllers; URL query string parser middleware.	Negative Test: Attempt to access admin UI routes without the <code>?is_admin=true</code> flag and assert an "Access Denied" (401/403) response.
ADM-2	Display a paginated list of all existing workflows with columns for Name, Description, Status, and Actions. Allow search and sort.	Admin UI Module (6.1); Business Logic Service (6.3); Data Access Module (6.4)	Workflow Dashboard UI component; Pagination controls; Search input field; GET /admin/workflows endpoint.	UI Test: Verify that the table renders correctly with all columns. Test search, sort, and pagination controls.
ADM-3	Allow admins to create a new workflow (Name required, Description optional). Name must be unique.	Admin UI Module (6.1); All Backend Modules (6.2, 6.3, 6.4)	Create Workflow modal form; POST /admin/workflows endpoint; CreateWorkflowAsync() service method.	Positive Test: Submit the create form and verify the new workflow appears in the list and exists in the database. Negative Test: Attempt to create a workflow with a duplicate name and assert a user-friendly

				validation error is shown.
ADM-4	Allow admins to update an existing workflow. No UI delete action for workflows.	Admin UI Module (6.1); Web Controller (6.2); Business Logic Service (6.3)	Edit Workflow modal form; PUT /admin/workflows/{id} endpoint.	Positive Test: Edit a workflow, save, and verify changes persist. Negative Test: Confirm there is no "delete" button for workflows in the UI. Manually attempt a DELETE request and assert a 405 Method Not Allowed response.
ADM-5	For a selected workflow, display its steps in a nested list with columns for StepNumber, StepName, etc.	Admin UI Module (6.1); Business Logic Service (6.3); Data Access Module (6.4)	Workflow Details component (nested list view); GET /admin/workflows/{id}/steps endpoint.	UI Test: Select a workflow and verify that a list of its correct steps is displayed. Verify all specified columns are present.
ADM-6 & 7	Create and update a workflow step dynamically. StepName is required and unique per workflow.	Admin UI Module (6.1); All Backend Modules (6.2, 6.3, 6.4)	Create/Edit Step modal form; POST & PUT endpoints for steps.	Positive Test: Add a new step to a workflow and verify it appears. Edit that step and verify changes persist. Negative Test: Attempt to create a step with a duplicate name within the same workflow and assert a validation error.
ADM-8	Delete a step from a workflow, with a confirmation dialog. Check for dependent tasks and reassign them to the next step if they exist.	Admin UI Module (6.1); Business Logic Service (6.3); Data Access Module (6.4)	Confirmation Dialog component; DELETE /admin/workflows/{wfid}/steps/{stepId}; DeleteWorkflowStepAsync() service method.	Integration Test: Execute delete on a step with active tasks. Verify the tasks are correctly reassigned to the next step's ID in the database and that the original step is deleted. Edge Case Test: Attempt to delete the final step of a

				workflow that has active tasks and assert a failure response.
ADM-9, 10, 11	Display tasks for a workflow. Allow admin to assign, unassign, or complete a task.	Admin UI Module (6.1); Business Logic Service (6.3); Data Access Module (6.4)	Task List view; Buttons for "Reassign" and "Complete"; endpoints like PUT /admin/tasks/{id}/assign.	Integration Test: Use the UI to reassign a task to a new user. Query the database directly to confirm the AssignedTo field was updated. Use the UI to complete a task and verify its StatusId is updated.
ADM-12	Display user-friendly errors for both client/server validation and for database exceptions (e.g., concurrency, FK violations).	All Modules	Global Exception Handling Middleware; Client-side form validation scripts; Notification/Toast Service in UI.	Negative Test (Concurrency): Simulate a concurrency conflict by updating a record in the DB directly, then try to save it via the UI. Assert that a "record has been modified" error appears. Negative Test (Validation): Submit a form with missing required fields and assert validation messages appear.
ADM-13	Display the task history for each step and/or workflow in a readable format.	Admin UI Module (6.1); Business Logic Service (6.3); Data Access Module (6.4)	History Tab UI component; TaskHistoryRepository class	UI Test: Select a workflow/step with a known history. Verify that the history records are displayed in correct chronological order.

11. Glossary

- DB – Database
- WFQ – Workflow Queue
- GUI – Graphical User Interface
- PHI – Protected Health Information
- PII – Personal Identifiable Information
- HITECH – Health Information Technology for Economic and Clinical Health Act
- HIPAA – Health Insurance Portability and Accountability Act
- RBAC – Role Based Access Control
- API – Application Programming Interface
- ADM - Administrator Requirement

Role Base Access control is a security measure. HIPAA and HITECH are federal regulations. Interfaces are a middleman between the software and end-user to help the user experience. API's allow us to use a service, such as SQL.

12. References

12.1 Internal Project Documents

R1. Workflow Queue (WFQ) — Software Requirements Specification (SRS); Authors: Project Team; Version: 1.2; Date: 10/23/2025; Location: .

R2. Workflow Queue Administrator Requirements; Authors: Sponsor/Project Team; Version: 1.0; Date: 10/8/2025; Location:

<https://docs.google.com/document/d/1k8LArYaEDdgyTWfOs79MFAGcLL8ZFRzioM-AsIEAfbE/edit?usp=sharing>.

12.2 Regulatory & Policy References

R3. U.S. HHS — Summary of the HIPAA Privacy Rule; Version: Current; Date: 2025-03-14; Location: <https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html>

R4. U.S. HHS — Summary of the HIPAA Security Rule; Version: Current; Date: 2024-12-30; Location: <https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html>.

R5. U.S. HHS — HITECH Act Enforcement Interim Final Rule; Version: Interim Final Rule; Date: 2017-06-16; Location: <https://www.hhs.gov/hipaa/for-professionals/special-topics/hitech-act-enforcement-interim-final-rule/index.html>.

R6. CMS — Emergency Medical Treatment & Labor Act (EMTALA); Version: Statutory Overview; Date: (page current); Location: <https://www.cms.gov/medicare/regulations-guidance/legislation/emergency-medical-treatment-labor-act>.

12.3 Protocols & Data Formats

R7. IETF — The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446; Date: 2018-08; Location: <https://datatracker.ietf.org/doc/html/rfc8446>.

R8. IETF — The Transport Layer Security (TLS) Protocol Version 1.2, RFC 5246; Date: 2008-08; Location: <https://www.rfc-editor.org/info/rfc5246>.

R9. IETF — The JavaScript Object Notation (JSON) Data Interchange Format, RFC 8259; Date: 2017-12; Location: <https://www.rfc-editor.org/info/rfc8259>.